Sort-O Technical Documentation

Plastic Sorting Robot
Sort-O

.

Project overview

The UR3e Plastic Sorter is an automated robotic platform designed to detect, classify, and sort objects based on their plastic material type. Leveraging a UR3e robotic arm, the system integrates computer vision and machine learning to identify plastics in real time and execute sorting actions, reducing the need for manual sorting in recycling workflows.

Key Features

- Automated OBB detection and classification using a fine-tuned YOLOv8 model, outputting plastic type, confidence score and rotation angle
- 3D localisation via RGB-D depth data and camera-to-robot calibration transform
- Machine learning-driven identification of plastic types (PET, HDPE, PP) with confidence scoring, from visual input.
- Robot manipulation via the ROS2 MoveIt implementation to sort objects into designated categories
- User interaction interface for system monitoring, configuration, manual override and emergency stop.

Subsystems

Subsystem	Lead	Description
Perception	Hyeokjun Jeong	Captures RGB-D data to detect plastic items, estimate 3D pose, and publish stable localisation data to the motion planner
Machine Learning	Rian Edwards	Detects and classifies plastic objects by type using a fine-tuned YOLOv8 OBB model, outputting class labels, confidence scores, 3D pose, orientation and a prioritised pick order via K-Means workspace planning
Interaction & Execution	Jarrel Redoblado	Manages the operator interface, system state machine, task coordination, and fault recovery logic
Motion Control & Planning	Bilguun Wicks	Plans and executes collision-free robot trajectories to pick and place objects into the correct sorting bins

Dependencies

Hardware

- **Robotic Arm:** Universal Robots UR3e
- **Gripper**: Onrobot RG2
- **Camera**: Intel Realsense RGB-D
- **Camera mount**: Tripod
- **Custom Tray:** with QR codes for calibration
- **Drop-off Bins**

Computing Requirements

- **Operating System:** Linux Ubuntu 22.04 LTS (Jammy Jellyfish)
- **Supported Setups**: Native Linux, Dual Boot, or WSL2

WSL2 may have limitations with USB device passthrough (e.g. RealSense camera) and network connectivity. Native Linux or dual boot is recommended for full hardware compatibility.

Software Dependencies

- **ROS 2:** Humble Hawksbill (see installation guide: Ubuntu (deb packages) — ROS 2 Documentation: Humble documentation)
- **Moveit2**: Moveit 2 Humble (see installation guide: Getting Started — MoveIt Documentation: Humble documentation)
- **Python 3.10** with: Ultralytics, pyrealsense2, opencv-python, scipy, numpy, scikit-learn

C++ Libraries:

- **Nlohmann_json:** Used for JSON parsing for configuration and messaging

Installation

Hardware

Environment

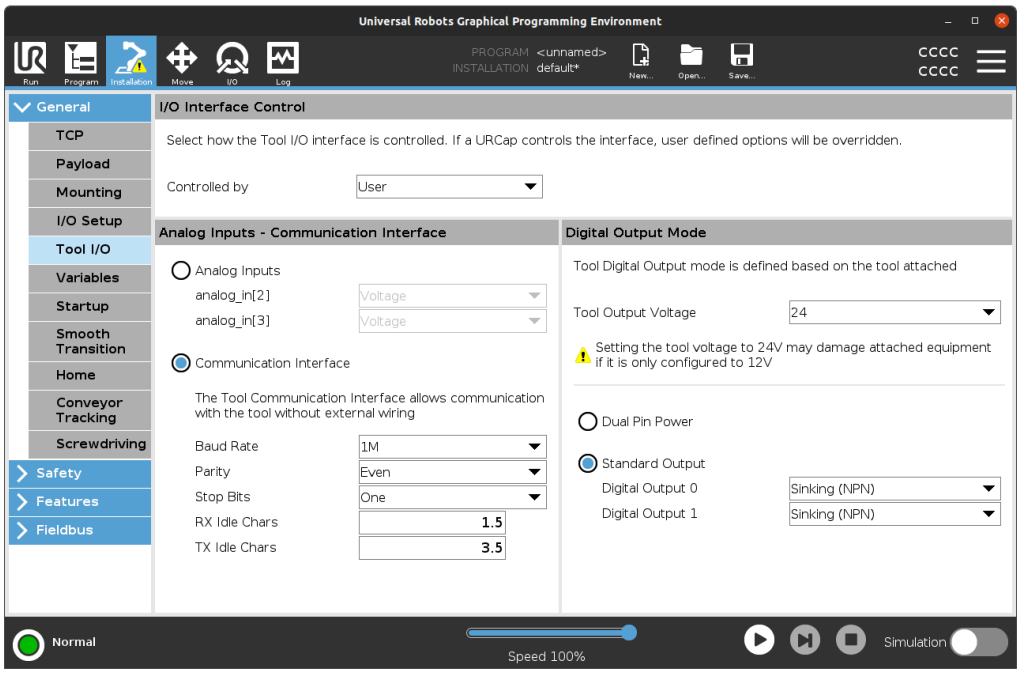
A black workspace board (approximately 500 mm × 320 mm) is placed on the trolley surface, acting as a bounded area to keep plastic objects stable and prevent them from sliding off. Four ArUco markers are positioned at each corner of the board, designated to be scanned which are numbered 1 to 4, used for camera calibration and spatial reference.

The Intel RealSense RGB-D camera is mounted on a tripod positioned to look straight down at the black workspace board.

A custom plywood bin assembly clips onto the handle of the metal trolley. It contains three square openings, each designated as a drop-off point for a specific plastic type (PET, HDPE, PP).

RG2 Gripper Setup

1. Connect the OnRobot Quick Changer to the Tool I/O port of the UR3e robot.
2. On the teach pendant, navigate to Installation → General → Tool I/O and configure the following parameters:



- Controlled by: User
- Communication Interface:
- Baud Rate: 1M

- Parity: Even
- Stop Bits: One
- RX Idle Chars: 1.5
- TX Idle Chars: 3.5
- Tool Output Voltage: 24V
- Standard Output:
- Digital Output 0: Sinking (NPN)
- Digital Output 1: Sinking (NPN)
- Usage

Still in the Installation tab, scroll down to URCaps → OnRobot and set the connection to "No Connection".

Software

1. **Clone the Repository**
   SSH:

   git clone: git@github.com:Jun-Je0ng/Robotics-Studio-2.git

   Or HTTPS:

   git clone https://github.com/Jun-Je0ng/Robotics-Studio-2.git

2. **Install ROS 2, MoveIt 2 and Python 3.10**
   If not already installed:

   ROS 2 Humble: Follow the official installation guide: Ubuntu (deb packages) — ROS 2 Documentation: Humble documentation
   MoveIt 2: Follow the Getting Started guide: Getting Started — MoveIt Documentation: Humble documentation)

3. **Install the UR Robot Driver**

   sudo apt-get install ros-humble-ur

4. **Connect to the Robot and Extract Calibration**
   4.1. Connect your computer to one of the UR3e robots in the MX Lab, then confirm connectivity:

   ping <robot_ip>

   4.2. Extract the calibration file (required for accurate operation):

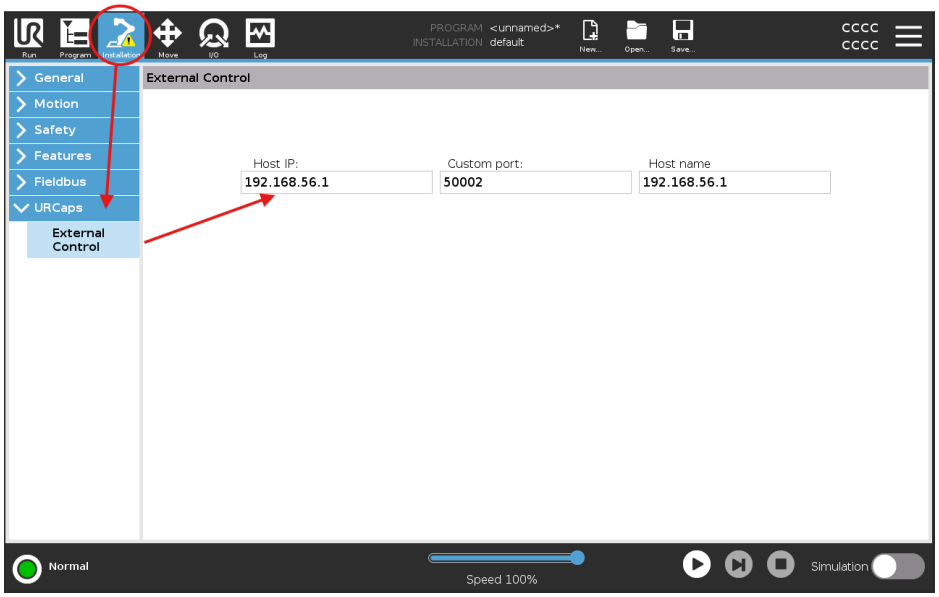
   *ros2 launch ur_calibration calibration_correction.launch.py *
   *robot_ip:=<robot_ip> *
   target_filename:="${HOME}/my_robot_calibration.yaml"

   4.3. Start the driver

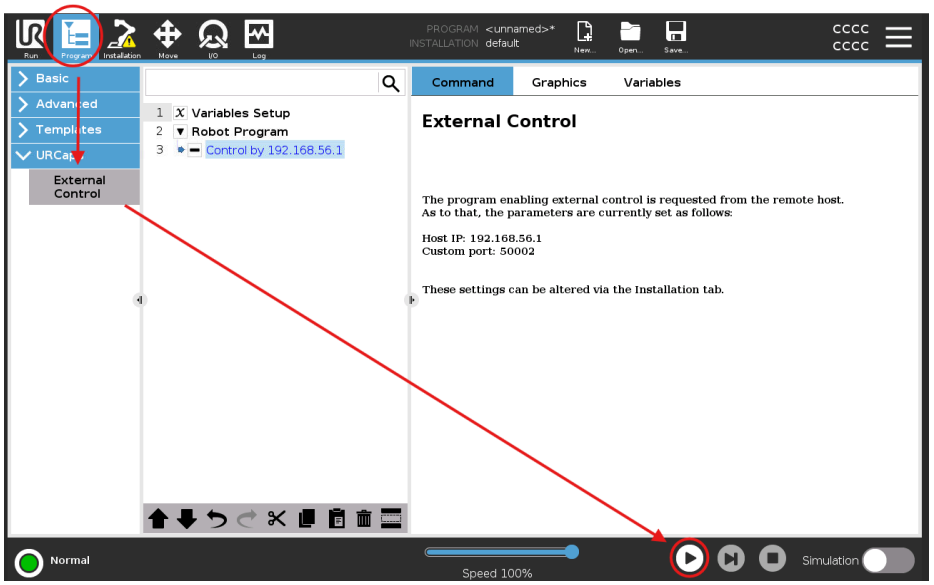
ros2 launch ur_onrobot_control start_robot.launch.py ur_type:=ur3e onrobot_type:=rg2 robot_ip:=<robot_ip>launch_rviz:=false

5. Setting up External Control on teach pendant and verify connection:

5.1. On the teach pendant, from the menu on the top of the screen, go to the Installation tab, scroll down to URCaps -> External Control and change the Host IP to your device IP.



5.2. Afterwards go back to the Program tab and add External Control to the list and press play.



If no errors appear, the robot is ready to receive commands from the computer. You may stop the driver and robot for now.

6. Install the UR + OnRobot ROS 2 Driver

Full documentation: [tonydle/UR_OnRobot_ROS2]

Navigate to your cloned workspace, then run the following:

 # Clone the UR_OnRobot driver into src

git clone https://github.com/tonydle/UR_OnRobot_ROS2.git src/ur_onrobot

 # Install git-managed dependencies

vcs import src --input src/ur_onrobot/required.repos --recursive

 # Install libnet for Modbus TCP/Serial (gripper comms)

sudo apt install libnet1-dev

 # Install ROS 2 dependencies

rosdep install -y --from-paths src --ignore-src

 # Build the workspace

colcon build --symlink-install

 # Source the workspace

source install/setup.bash

7. **Verify the Installation**
   Launch the robot visualisation to confirm the URDF loads correctly with the gripper attached:

   ros2 launch ur_onrobot_description view_robot.launch.py ur_type:=ur3e onrobot_type:=rg2

Running the system

Launch commands

Prerequisites

- Robot reachable at `192.168.0.XXX`
- ROS2 workspace sourced (`~/ros2_ws`)
- All packages built: `ur_onrobot_control`, `ur_gripper_demo`, `plastic_detection`

Launch Sequence

Run each command in a separate terminal in the order listed. Wait for each step to fully initialise before proceeding to the next.

Terminal 1 — Serial Bridge

```
socat pty,link=/tmp/ttyUR,raw,ignoreeof tcp:192.168.0.XXX:54321
```

Sets up the serial-over-TCP bridge to the UR robot. Must be running before any ROS2 nodes start.

Terminal 2 — Robot Hardware & Control Stack

```
ros2 launch ur_onrobot_control start_robot.launch.py \
  ur_type=ur3e \
  onrobot_type=rg2 \
  robot_ip=192.168.0.XXX \
  use_fake_hardware=false
```

Brings up the UR3e robot driver and OnRobot RG2 gripper hardware interface.

Terminal 3 — MoveIt + Gripper Demo

```
ros2 launch ur_gripper_demo ur_moveit.launch.py \
  ur_type=ur3e \
  onrobot_type=rg2 \
  demo_type=reactive \
  sim=false
```

Launches MoveIt motion planning with reactive gripper demo mode.

Terminal 4 — Camera Calibration

```
python3 camera_calibration.py
```

Run camera calibration before starting any vision-dependent nodes.

Terminal 5 — Status GUI

```
ros2 run ur_gripper_demo status_gui
```

Opens the gripper/robot status monitoring GUI.

Terminal 6 — OBB Detector

```
ros2 run plastic_detection obb_detector_node
```

Starts the oriented bounding box detector for plastic object detection.

Terminal 7 — Workspace Planner

```
ros2 run plastic_detection workspace_planner_node
```

Starts the workspace planner. Depends on OBB detector output — ensure Terminal 6 is running first.

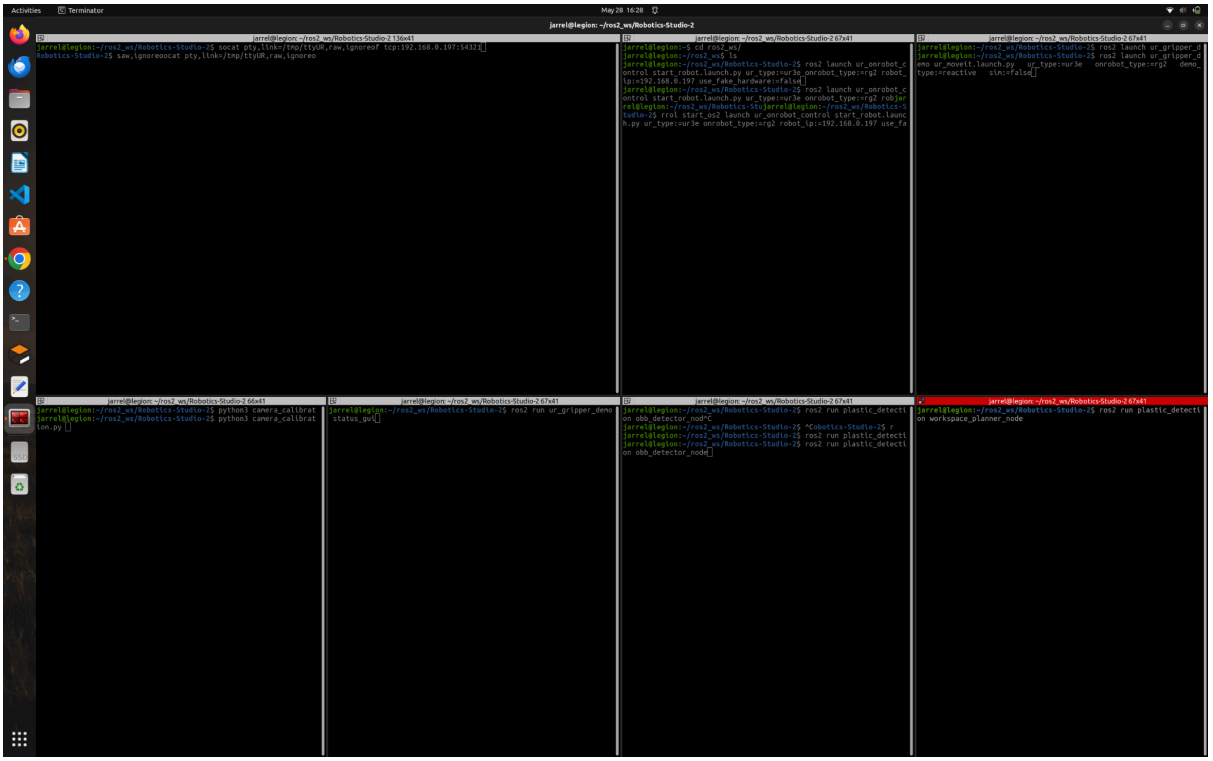
Summary Table

Terminal	Node/Script	Purpose
1	`socat`	Serial bridge to robot
2	`start_robot.launch.py`	Robot hardware & control
3	`ur_moveit.launch.py`	MoveIt + gripper
4	`camera_calibration.py`	Camera calibration
5	`status_gui`	Status monitoring
6	`obb_detector_node`	Object detection
7	`workspace_planner_node`	Workspace planning

Setting Drop Pose (Before Starting)

Before pressing START in the GUI:

1. Use Freedrive on the teach pendant to move the robot to the desired drop pose for each bin.
2. Send the pose via the GUI using `SAVE_bin buttons`



Subsystem specifics

Perception

Purpose

The Perception subsystem detects plastic objects on the work surface, determines their 3D position, orientation, and physical dimensions in the robot's coordinate frame, and publishes this information as structured ROS2 messages for downstream pick-and-place execution. It uses a top-down RGB-D camera combined with a trained YOLO OBB (Oriented Bounding Box) model to classify and locate objects, then transforms detection results from image space into robot-frame 3D coordinates using a pre-calibrated extrinsic transform matrix.

Key topics / Services / Files / Nodes

Nodes

Node	FIle	Role
obb_detector_node	*src/plastic_detection/plastic_detection/obb_detector_node.py*	Runs YOLO OBB inference on live camera feed, computes 3D pose, applies filtering and stability gating, publishes JSON detections
plastic_detections_translator	*src/ur_gripper_demo/scripts/plastic_detections_translator.py*	Converts raw JSON detections into *object_msgs/ObjectArray* ROS messages with correct coordinate frame transformations

Key topics

Topic	Type	Direction	Description
/plastic_detections	*std_msgs/String* (JSON)	Published by obb_detector_node	Array of stable object detections with pose, dimensions, classification, and debug info
/perception/objects	*object_msgs/ObjectArray*	Published by plastic_detections_translator	Structured ROS message consumed by the pick-and-place C++ node

File

File	Description
src/plastic_detection/plastic_detection/obb_detector_node.py	Main perception node
src/plastic_detection/plastic_detection/best.pt	Trained YOLO OBB model weights
src/plastic_detection/plastic_detection/camera_to_robot_calibration.json	4×4 extrinsic transform matrix (camera frame → robot base_link frame)
src/ur_gripper_demo/scripts/plastic_detections_translator.py	Message translator node

Inputs and outputs

Inputs

- Intel RealSense D4xx RGB stream: 640×480 @ 30fps, BGR8 format
- Intel RealSense D4xx Depth stream: 640×480 @ 30fps, Z16 format (aligned to color frame)
- *camera_to_robot_calibration.json*: pre-calibrated 4×4 homogeneous transform matrix
- *best.pt*: YOLO OBB model trained on plastic waste classes

Output

```
{
  "instance_id": "pet_bottle_0",
  "pose": {
    "position": { "x": 120.3, "y": -45.1, "z": 62.0 },rr
    "orientation": { "qx": 0.0, "qy": 0.0, "qz": 0.383, "qw": 0.924 }
  },
  "dimensions": { "dx_mm": 65.2, "dy_mm": 32.1 },
  "classification": { "class": "pet           _bottle", "confidence": 0.91 },
  "debug": {
    "z_table_mm": 12.0,
    "z_approach_mm": 162.0,
    "angle_deg": 45.23,
    "angle_rad": 0.7896,
    "depth_m": 0.4123,
    "dz_source": "depth"
  }
}
```

How to run or test the subsystem independently

Prerequisites:

- Intel RealSense camera plugged in via USB 3
- ROS2 Humble sourced: source /opt/ros/humble/setup.bash
- Workspace built and sourced: source install/setup.bash (from workspace root)

Run the detector:

```
ros2 run plastic_detection obb_detector_node
```

A camera window appears showing live detections with bounding boxes. Status shows "Stabilising..." (orange) until 5 consecutive frames, then "PUBLISHING" (green).

Run the translator (separate terminal):

```
ros2 run ur_gripper_demo plastic_detections_translator
```

Verify output:

```
# Raw JSON from detector
```

```
ros2 topic echo /plastic_detections

# Structured ObjectArray from translator
ros2 topic echo /perception/objects

# Check publish rate
ros2 topic hz /plastic_detections
```

Test without robot: Both nodes run fully independently of the robot and MoveIt. Place a plastic object under the camera and verify position values update correctly in the terminal output.

Configurable settings or important parameters

Parameter	Default	Description
STABILITY_FRAMES	5	Consecutive frames required before publishing
SMOOTHING_WINDOW	5	Number of frames averaged for position smoothing
DEPTH_KERNEL_SIZE	11	Pixel kernel size for median depth sampling
DEPTH_JUMP_THRESHOLD	0.05 m	Maximum frame-to-frame depth change before spike rejection
GRIP_OFFSET	50 mm	Height offset above table for grip pose Z
APPROACH_OFFSET	150 mm	Height offset above table for approach pose Z
SHOW_DISPLAY	True	Set to False to disable the OpenCV camera window
MIN_CONFIDENCE	0.50	Secondary confidence filter in translator
GRIPPER_TCP_OFFSET_M	0.180 m	Offset from tool0 frame to gripper fingertip

Known limitations or assumptions

- **Top-down camera only:** The height measurement (compute_height_from_depth) assumes the camera views the work surface from above. Angled views would produce incorrect height readings.

- **Flat table surface:** The work plane model assumes a roughly flat surface. Significant table tilt or curved surfaces will affect position accuracy.

- **Height measurement below ~8mm:** The RealSense depth noise floor (~3–5mm) approaches the object height for very flat objects, making reliable height measurement impossible.

- **Reflective/transparent objects:** Clear bottles and shiny metal surfaces cause depth drop-outs. The median kernel and spike filter mitigate this but cannot fully correct it.

- **Fixed calibration:** The camera-to-robot transform is computed once and stored. If the camera is physically moved, recalibration is required.

- **Single camera:** Occlusion (one object hidden behind another) will result in the hidden object not being detected.

Machine Learning

Purpose

The machine learning subsystem is responsible for real-time detection, classification and orientation estimation of plastic bottles in the robot workspace. It processes raw RGB-D camera frames, identifies plastic objects by visual appearance (shape, colour and form factor), computes their 3D pose in robot base frame coordinates, and publishes structured data over ROS2 for downstream motion planning and gripper control.

Key Files and Nodes

obb_detector_node.py
Core detection node. Loads the trained YOLOv8 OBB model, runs inference on live RealSense frames, applies confidence and stability filtering, tracks each object instance independently using pixel-radius matching, converts pixel detections to 3D robot frame coordinates via homography or ray intersection, and publishes to /plastic_detections. Also publishes annotated camera frames to /camera/annotated for the GUI.

workspace_planner_node.py
Pick ordering node. Subscribes to /plastic_detections, runs K-Means clustering on detected object XY positions, orders clusters by proximity to the robot base, orders objects within each cluster by distance to robot, and publishes a prioritised pick order to /pick_queue.

best.pt
Trained YOLOv8n-OBB model weights. Fine-tuned from COCO pretrained weights on 449 custom plastic bottle images across 3 classes. Download from repository or this link:

https://drive.google.com/file/d/131pSWHQh9DssEY7xiMnQ55VRqxLlvITJ/view?usp=drive_link

camera_to_robot_calibration.json
Calibration file containing the 4x4 transform matrix, homography matrix for pixel to robot mm mapping, and calibration point coordinates. Generated by camera_calibration.py. This is mentioned more in the Perception subsystem.

OBB_dataset/data.yaml
Dataset configuration file. Defines training and validation image paths, number of classes and class names.

Inputs and Outputs

Inputs:

Source	Type	Description
RealSense camera	RGB frame	640x480 BGR image at 30fps used for OBB inference
RealSense camera	Depth frame	640x480 aligned depth frame used for 3D deprojection
best.pt	File	Trained model weights loaded at node startup
camera_to_robot_calibration.json	File	Camera to robot transform matrix loaded at node startup

Outputs:

Topic	Type	Description
/plastic_detections	std_msgs/String	JSON array of stable detections including instance ID, position, orientation quaternion, dimensions, class and confidence
/pick_queue	std_msgs/String	JSON array of detections with added pick_order, cluster_id, cluster_priority and dist_to_robot_mm fields
/camera/annotated	sensor_msgs/Image	Annotated camera frame with OBB boxes, labels, stability status and calibration overlays published for the GUI

/plastic_detections message format:

[{ "instance_id": "pet_bottle_0",
"pose": {
"position": {"x": 312.4, "y": -88.1, "z": -142.6},
"orientation": {"qx": 0.0, "qy": 0.0, "qz": 0.383, "qw": 0.924}},
"dimensions": {"dx_mm": 88.2, "dy_mm": 241.5, "dz_mm": 72.1},
"classification": {"class": "pet_bottle", "confidence": 0.971},
"debug": {
"angle_deg": 45.2,
"depth_m": 0.614,
"dz_source": "depth"}}]

How to Run and Test Independently

Prerequisites -- install dependencies:

pip install ultralytics pyrealsense2 opencv-python scipy numpy scikit-learn

Download model weights from Google Drive and replace the path in script:

/home/rian/ml_project/runs/obb/train/weights/best.pt

Build the ROS2 package:

colcon build --packages-select plastic_detection

source install/setup.bash

Run each node in a separate terminal:

 # Terminal 1

ros2 run plastic_detection obb_detector_node

 # Terminal 2

ros2 run plastic_detection workspace_planner_node

 # Terminal 3

Verify outputs by echoing topics:

ros2 topic echo /plastic_detections

ros2 topic echo /pick_queue

Ros2 topic echo/camera/annotated

 # Terminal 4

Test without full ROS2 stack: The detector can be run standalone without ROS2 using obb_pose_detector.py. This runs detection and prints results to the terminal without publishing any topics:

python3 obb_pose_detector.py

Extending the Model to Additional Plastic Classes

This guide explains how to scale the plastic detection model from the current 3 classes (HDPE, PET,PP) to 4 or more classes. The pipeline is designed to be extensible, adding a new class requires only data collection, labelling and retraining.

Step 1: Collect Images

Capture 80 to 150 images of the new plastic type using the RealSense camera. Cover multiple orientations, positions, distances and lighting conditions. Save images to the dataset folder.

Step 2: Label in Label Studio

Start Label Studio:

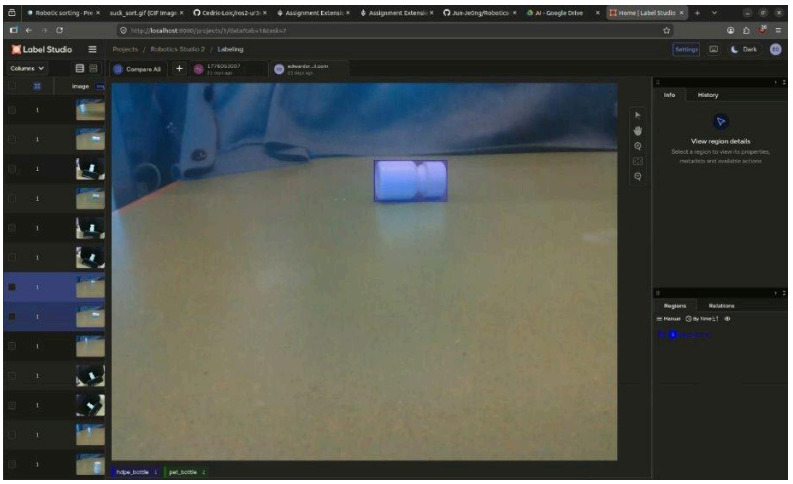
source ~/label-studio-env/bin/activate

export LABEL_STUDIO_DATA_UPLOAD_MAX_NUMBER_FILES=1000

label-studio --host 0.0.0.0 --port 8080

Create a new project, add the new class to the labelling interface with canRotate=true, annotate with rotated bounding boxes, and export as YOLOv8 OBB format.

Upload the new images and annotate each one with a rotated bounding box.



After drawing the bounding box for each image, export as YOLOv8 OBB format.

Step 3: Fix Class ID

Label studio assigns class ID 0 by default. Remap to the correct ID (3 for a fourth class):

```
python3 << 'EOF'
import os
labels_dir = "/path/to/exported/labels"
NEW_CLASS_ID = 3
for label_file in os.listdir(labels_dir):
    if not label_file.endswith(".txt"):
        continue
    path = os.path.join(labels_dir, label_file)
    with open(path, "r") as f:
        lines = f.readlines()
    new_lines = []
    for line in lines:
        parts = line.strip().split()
        if parts:
            parts[0] = str(NEW_CLASS_ID)
            new_lines.append(" ".join(parts))
    with open(path, "w") as f:
        f.write("\n".join(new_lines))
print(f"Remapped all labels to class ID {NEW_CLASS_ID}")
EOF
```

Step 4: Merge into Existing Dataset

Split the new images 80/20 and copy into the existing train/val folders:

```
python3 << 'EOF'
import os, shutil, random
new_images = "/path/to/exported/images"
new_labels = "/path/to/exported/labels"
train_images = "/home/rian/ml_project/OBB_dataset/train/images"
train_labels = "/home/rian/ml_project/OBB_dataset/train/labels"
val_images   = "/home/rian/ml_project/OBB_dataset/val/images"
val_labels   = "/home/rian/ml_project/OBB_dataset/val/labels"
files = [f for f in os.listdir(new_images) if f.lower().endswith((".jpg", ".jpeg", ".png"))]
random.shuffle(files)
split = int(len(files) * 0.8)
for f in files[:split]:
    shutil.copy(os.path.join(new_images, f), os.path.join(train_images, f))
    lf = os.path.splitext(f)[0] + ".txt"
    if os.path.exists(os.path.join(new_labels, lf)):
        shutil.copy(os.path.join(new_labels, lf), os.path.join(train_labels, lf))
for f in files[split:]:
    shutil.copy(os.path.join(new_images, f), os.path.join(val_images, f))
    lf = os.path.splitext(f)[0] + ".txt"
    if os.path.exists(os.path.join(new_labels, lf)):
        shutil.copy(os.path.join(new_labels, lf), os.path.join(val_labels, lf))
print(f"Merged {len(files[:split])} train, {len(files[split:])} val images")
EOF
```

Step 5: Update data.yaml

Add the new class name and increment nc:

train: /home/rian/ml_project/OBB_dataset/train/images

val: /home/rian/ml_project/OBB_dataset/val/images

nc: 4

names: ['hdpe_bottle', 'pet_bottle', 'pp_container', 'pvc_bottle']

The class order must match the class IDs exactly: index 0 is hdpe_bottle, index 1 is pet_bottle, index 2 is pp_container, index 3 is the new class.

Step 6: Retrain

Retrain from the existing best.pt weights rather than from scratch. This is transfer learning, the model already knows how to detect the first 3 classes and adapts to include the new one:

```
cd ~/ml_project

yolo obb train \
  data=/home/rian/ml_project/OBB_dataset/data.yaml \
  model=/home/rian/ml_project/runs/obb/pet_hdpe_pp_v1/weights/best.pt \
  epochs=50 \
  imgsz=640 \
  hsv_h=0.015 hsv_s=0.7 hsv_v=0.4 \
  degrees=45.0 translate=0.1 scale=0.5 shear=15.0 \
  mosaic=1.0 mixup=0.1 copy_paste=0.1 \
  name=extended_4class
```

Step 7: Rebuild and restart, Update MODEL_PATH in obb_detector node.py to point to the new weights

Configurable Parameters

obb_detector_node.py

Parameter	Default	Description
MODEL_PATH	...best.pt	Path to trained YOLOv8 OBB weights
CALIBRATION_FILE	...camera_to_robot_calibration.json	Path to camera-robot transform JSON
CONF_THRESHOLD	0.7	Minimum confidence to accept a detection
STABILITY_FRAMES	5	Consecutive frames required before publishing
MATCH_RADIUS_PX	60	Pixel radius for matching detections to existing instance tracks
DEPTH_KERNEL_SIZE	11	Pixel patch size for median depth sampling
SMOOTHING_WINDOW	5	Rolling window size for XY position smoothing
DEPTH_JUMP_THRESHOLD	0.05	Maximum allowed depth change between frames in metres
GRIP_OFFSET	50	Z height above table surface for grip point in mm
APPROACH_OFFSET	150	Z height above table surface for hover point in mm
SHOW_DISPLAY	True	Enables live camera window. Comment out to disable

workspace_planner_node.py

Parameter	Default	Description
ROBOT_BASE_X	0.0	Robot base X position in robot frame mm used for cluster distance scoring
ROBOT_BASE_Y	0.0	Robot base Y position in robot frame mm used for cluster distance scoring
n_clusters	min(len(detections), 3)	Number of K-Means clusters, capped at object count

Known Limitations and Assumptions

Calibration accuracy
The current camera-to-robot calibration was computed from 4 ArUco marker points. If the homography matrix is not present in the calibration JSON the node falls back to 3D ray intersection which may have scale error. Recalibration with 6 or more points and regeneration of the homography is recommended for accurate pick-and-place.

Depth measurement reliability
The RealSense camera is mounted at approximately 40 degrees. At this angle, depth readings at workspace edges become noisy. The bottle diameter measurement (dz_mm) uses a centre-to-edge depth comparison which can fail under these conditions. The node falls back to a pixel-based estimate and omits dz_mm from the message if depth measurement is unreliable.

Fixed workspace assumption
The work plane Z formula (get_table_z) assumes the workspace surface is fixed and planar. If the table is moved or the camera position changes, the calibration JSON and work plane coefficients must both be recomputed.

K-Means with single object
When only one object is detected K-Means is skipped entirely. The single object is assigned pick_order=1 and dist_to_robot_mm computed directly without clustering.

Same class multiple instances
The instance tracker uses pixel proximity to distinguish two bottles of the same class. If two bottles of the same class are very close together (within MATCH_RADIUS_PX pixels) they may be assigned the same track key and treated as one object.

Interaction & Execution

Purpose

The GUI subsystem is the operator-facing control layer. It sends commands to the motion controller via /motion_system/command, monitors all key system topics for display, saves bin drop-off poses by reading the live TF2 tree, and provides an E-Stop that also force-kills the motion controller process as a hard fallback.

Key Features / Subsystem List

Component	File	Role
Control Panel GUI	scripts/status_gui.py	Operator interface — start/stop, gripper control, bin pose teaching, live camera and pick queue display
Reactive Motion Controller	**src/motion_controller_reactive.cpp**	MoveIt-based pick-and-place execution — grasp strategy selection, path planning, drop detection
Gripper Bridge	**scripts/gripper_bridge.py**	Real-robot only — translates GripperCommand actions to raw finger-width controller commands
Bin Poses Config	**config/bin_poses.json**	Per-class drop-off poses taught via the GUI and read by the motion controller at runtime
Master Launch File	**launch/ur_moveit.launch.py**	Starts MoveIt, motion controller, GUI, gripper bridge, and perception translator together
Camera Calibration	**plastic_detection/camera_calibration.py**	One-time setup — detects 4 ArUco markers on tray corners to compute and save the camera→robot transform (camera_to_robot_calibration.json)

Dependencies

Hardware

Item	Purpose
UR3e robot arm	Motion execution
OnRobot RG2 gripper	Grasping
Intel RealSense RGB-D camera	Perception (feeds/ camera/annotated)
Workbench/ sorting tray with guard rails	Workspace
Laptop/ workstation	ROS 2 host, connected to UR3e via Ethernet.

Computing Specs

- OS: Ubuntu 22.04 (x86_64)
- Network: Laptop on 192.168.0.x subnet, UR3e controller at a fixed IP (e.g. 192.168.0.194)
- GPU: Not required for this subsystem; GPU recommended for the OBB detector node

Software

Package	Version/Notes
ROS2	Humble
Python	3.10+
tkinter	stdlib
Pillow	Pip install Pillow
Numpy	Pip install numpy
Rclpy	ROS 2 Python client -installed with ROS
Tf2_ros	For TF2 EEF pose lookup (bin saving)
Control_msgs	GripperCommand action type
MoveIt 2	Humble — moveit_ros_move_group, moveit_servo

Installation

Hardware

1. Mount the RealSense camera above the workspace with a clear top-down view of the sorting tray.
2. Connect the camera via USB 3.
3. Connect the laptop to the UR3e controller box via Ethernet; set laptop IP to 192.168.0.x.
4. Attach the RG2 gripper to the UR3e flange and connect the gripper cable.

Software

```
# 1. Clone the repository
git clone https://github.com/Jun-Je0ng/Robotics-Studio-2.git
cd Robotics-Studio-2

# 2. Install Python dependencies
pip install Pillow numpy

# 3. Install ROS dependencies
rosdep install --from-paths src --ignore-src -r -y

# 4. Build
colcon build --packages-select ur_gripper_demo plastic_detection
source install/setup.bash
```

Three terminals are required:

```
# Terminal 1 — UR driver + MoveIt (real robot, replace IP as needed)
ros2 launch ur_gripper_demo ur_moveit.launch.py

# Terminal 2 — OBB detector (camera + YOLO perception)
ros2 run plastic_detection obb_detector_node

# Terminal 3 — Workspace planner (cluster-based pick ordering)
ros2 run plastic_detection workspace_planner_node

# Terminal 4 — GUI
source install/setup.bash
ros2 run ur_gripper_demo status_guiRunning the System
```

Running the System
One-Time Setup — Camera Calibration
Before running the system for the first time (or after repositioning the camera or tray), you must calibrate the camera-to-robot transform.

Step 0a — Measure tray corner positions (once ever)

Jog the UR3e EEF directly above each ArUco marker corner on the tray and record the robot-frame XYZ from the teach pendant (or ros2 topic echo /tf). Update TRAY_CORNERS_ROBOT_M in plastic_detection/camera_calibration.py:

```
TRAY_CORNERS_ROBOT_M = {
    0: (x, y, z),   # front-right corner
    1: (x, y, z),   # back-right corner
    2: (x, y, z),   # front-left corner
    3: (x, y, z),   # back-left corner
}
```

Step 0b — Run calibration

Place the tray in its fixed position with all 4 ArUco markers visible, then run:

```
cd src/plastic_detection/plastic_detection
python3 camera_calibration.py
```

A live camera feed opens. When all 4 markers are detected and stable (green = READY), the script auto-captures and saves the transform. You can also press SPACE to capture manually or Q to quit without saving.

The output file camera_to_robot_calibration.json is written to the same directory and is read automatically by the OBB detector and perception translator at runtime. Re-run this step any time the camera or tray is moved.

Real Robot — Step by Step

Step 1 — Turn on the UR3e

- Release brakes on the teach pendant.
- Go to Installation → URCaps → External Control.
- Enter the laptop IP for both Hostname and IP fields.
- Ensure Simulation mode is OFF.
- Press ▸ Play on the teach pendant.

Step 2 — Start the UR driver

ros2 launch ur_robot_driver ur_control.launch.py ur_type:=ur3e robot_ip:=192.168.0.194 launch_rviz:=true

Step 3 — Launch the full system

source install/setup.bash
ros2 launch ur_gripper_demo ur_moveit.launch.py demo_type:=reactive sim:=false robot_ip:=192.168.0.xxx

Step 4 — Start the perception nodes (separate terminal)

source install/setup.bash
ros2 run plastic_detection obb_detector_node

Simulation (URSim)

Terminal 1 — Start URSim
ros2 run ur_client_library start_ursim.sh -m ur3e
Open Polyscope at http://192.168.56.101:6080/vnc.html, press ▸ Play

Terminal 2 — UR driver (simulator IP)
ros2 launch ur_robot_driver ur_control.launch.py ur_type:=ur3e robot_ip:=192.168.56.101

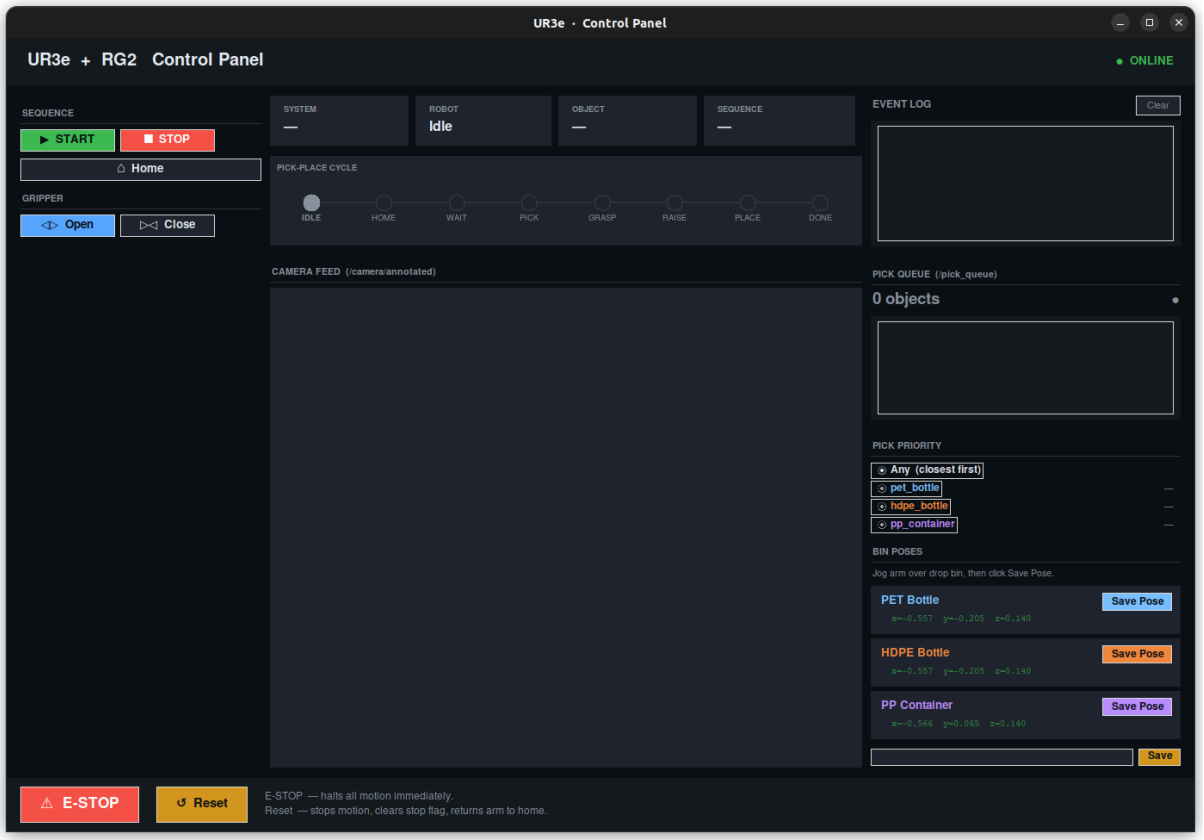
Terminal 3 — Full system in sim mode
ros2 launch ur_gripper_demo ur_moveit.launch.py demo_type:=reactive sim:=true

Launch Arguments

Argument	Default	Options	Desciption
demo_type	demo	reactive,demo,pick_place,none	Which controller to launch
sim	False	true/false	Disables gripper stall detection in sim
robot_ip	192.168.0.xxx	any ip	UR3e controller IP
ur_type	Ur3e	urxx	Robot model
onrobot_type	rg2	Rg2, rg6	Gripper model
launch_rviz	true	true/false	Open RViz
launch	true	true/false	Start MoveIt Servo

Window Layout

A 1380 × 920 px dark-themed control panel appears. The robot homes, then waits in WAITING state for perception to publish objects. When objects appear in /pick_queue the controller enters PREGRASP and begins picking.



Subsystem Specifics

1. **Control Panel GUI (scripts/status_gui.py)**

Controls Reference
Sequence (left panel)

Button	**Command sent**	**Effect**
Start	**Start**	**Begins the reactive pick and place cycle**
Stop	**Stop**	**Stops after the current pick completes; does not interrupt mid-cycle**
Home	**Home**	**Returns the arm to the up named pose**

Gripper (left panel)

Button	**Command sent**	**Effect**
Open	**open_gripper**	**Opens the RG2 to 110 mm**
Close	**Close_gripper**	**Closes the RG2 to near-zero**

Poses (left panel)
Forwards save_pose, save_poses_file, load_poses, and clear_poses commands to the motion controller's handleCommand. These are placeholders — implement the handlers in motion_controller_reactive.cpp if needed.

Pick Priority (right panel)
Radio buttons set which bottle type the arm prefers to pick first.

Selection	Command sent
Any (closest first)	TARGET_CLASS: (empty)
pet_bottle	TARGET_CLASS:pet_bottle
hdpe_bottle	TARGET_CLASS:hdpe_bottle

Live confidence scores (from /plastic_detections) are shown next to each type. The confidence display updates every detection frame; — means the type is not currently detected.

Bin Poses (right panel)
Used to teach the drop-off position for each bottle type.

1. Jog the arm so the gripper is centred over the correct bin.
2. Click the type button (e.g. pet_bottle) to save the current end-effector pose to config/bin_poses.json.
3. ↺ Reload Bins re-reads the JSON without restarting the controller.

E-STOP bar (bottom)

Button	Command sent	Effect
E-STOP	estop	Calls arm.stop() immediately; sets stop flag
Reset	Reset	Calls arm.stop(), clears stop flag, requests home

Pick-Place Cycle Diagram (middle panel)

● IDLE ○ HOME ○ WAIT ○ PICK ○ GRASP ○ RAISE ○ PLACE ○ DONE

Node colour	Meaning
Grey (filled)	IDLE - waiting for START
Blue	HOMING - arm returning to home
Cyan	WAITING - waiting for perception framery
Yellow	PREGRASP - planning and moving above object
Orange	GRASPING - descending and closing gripper
Cyan	RAISING - lifting object
Cyan	TRANSPORTING - moving to bin and releasing
Green	DONE - cycle complete
Red	FAILED - gripper found nothing; will ret

Past nodes dim; the current node is filled bright. Resizes automatically with the window.

Event Log (middle panel)
Colour-coded timestamped log of every received event and every command the GUI sends. Click Clear to wipe it. Colours:

Colour	Meaning
Green	Success
Yellow	Warning
Red	Error/Failure
Blue	GUI - sent command
White	Informational

Camera Feed (right panel)
Subscribes to /camera/annotated (sensor_msgs/Image, bgr8 encoding). Displays the annotated frame from the OBB detector node at 336 × 252 px. Shows "Waiting for camera..." until the first frame arrives.

Published Topics

Topic	Type	Source
/motion_system/command	std_msgs/String	All control commands

Subscribed Topics

Topic	Type	Source
/system_status	std_msgs/String	Motion controller
/robot_state	std_msgs/String	Motion controller
/event_log	std_msgs/String	Motion controller
/motion_system/status	std_msgs/String	Motion controller
/motion_system/current_object	std_msgs/String	Motion controller
/plastic_detections	std_msgs/String(JSON)	Obb detector node
/pick_queue	std_msgs/String(JSON)	Workspace planner node
/camera/annotated	sensor_msgs/Image	Obb detector node

/plastic_detections is used as a fallback display source; once /pick_queue publishes its first message the GUI switches to that enriched view permanently for the session.

Commands Reference

Button	Command Published	Effect
START	start	Begins reactive pick-and-place cycle
STOPs	stop	Stops after current pick completes
Home	home	Returns arm to up named pose
Open	Open_gripper	Opens RG2

Inputs and Outputs
Inputs:

- Live system, robot, sequence, and object status from the motion node
- Live detection list with class, confidence, position (mm), and dimensions from the camera
- Live grip pose list with grip position, jaw opening, and approach direction

Outputs:

Button	Command string sent	Effect
Start	Start	Begin pick-and-place sequence
E-STOP	Estop+ pkill -9 gripper_pick_place	Stop arm + kill process at OS levl
Reset	Reset	Clear stop flag
Home	Home	Move arm to home position
Open	Open_gripper	Open gripper to 110 mm
Close	close_gripper	Close gripper to 10 mm
Cartesian jog buttons	Jog_cartesian <dx> <dy> 0 0 0	Continuous jog at 10 Hz while held

How to Run
Normal run:
source ~/ros2_ws/Robotics-Studio-2/install/setup.bash
ros2 run ur_gripper_demo status_gui

Test the GUI without the robot running — open the GUI on its own and inject fake status messages from a second terminal to verify the display updates correctly:

Inject a fake system status
ros2 topic pub --once /system_status std_msgs/msg/String "{data: 'Running'}"

Inject a fake robot state
ros2 topic pub --once /robot_state std_msgs/msg/String "{data: 'Picking'}"

Inject a fake event log entry
ros2 topic pub --once /event_log std_msgs/msg/String "{data: 'Picked object plastic'}"

Watch what commands the GUI sends when you press buttons
ros2 topic echo /motion_system/command

Test the E-STOP — with gripper_pick_place running, press the E-STOP button and confirm:

1. The button turns dark red and locks
2. The terminal running gripper_pick_place exits
3. The SYSTEM card shows E-STOPPED

Configurable Settings or Important Parameters

Setting	Location	Default	Description
Jog step size	GUI spinbox	10mm	30% of max speed - increased carefully
Jog publish rate	_jog_worker	10 Hz	30% of max acceleration
ROS spin rate	_spin_ros	100 ms	Max MoveIt is allowed to plan
Window size	_build	1380x800, non-resizable	Fixed layout size

TF2

The GUI listens for the transform base → gripper_tcp when saving bin poses. Both robot_state_publisher and MoveIt must be running for this lookup to succeed.

Known Limitations and Assumptions

- Hardcoded absolute paths — BIN_POSES_PATH and BIN_POSES_INSTALL_PATH are tied to /home/jarrel/.... The GUI will silently fail to save bin poses on any other machine or if the workspace is moved.

- 10 Hz GUI update ceiling — rclpy.spin_once is called every 100 ms. Camera frames and detections published faster than 10 Hz are silently dropped in the GUI, even though the ROS subscriber queue depth would allow buffering them.

- E-STOP is a hard kill — pkill -9 (SIGKILL) gives the motion controller no chance to execute a graceful shutdown (e.g., completing a safe stop trajectory). The arm stops wherever it is at the moment of the kill.

- Open/Close bypass the motion controller — The Gripper buttons send directly to the action server, independently of the controller's internal state. Pressing Open or Close mid-pick can conflict with a grasp in progress.

- TF2 must be live to save bin poses — If robot_state_publisher or MoveIt is not running, the TF2 lookup in _get_eef_pose() will time out and the save is aborted. The

SAVE_BIN command is still sent to the controller, which may succeed on its own TF2 lookup.

- /pick_queue switching is one-way per session — Once any message arrives on /pick_queue, the enriched display mode is locked in for the rest of the session (_pick_queue_active flag). If workspace_planner_node crashes, the pick queue display stalls rather than falling back to raw /plastic_detections.

- Camera encoding assumed BGR(A) — The frame callback checks the number of channels but does not validate the encoding field of the image message. Images encoded as rgb8 or mono8 will display with wrong colours or crash the reshape.

- No namespace support — All topic names are absolute. Running two instances of the GUI simultaneously will double-publish commands to the controller.

- open_gripper / close_gripper string commands are placeholder stubs in the motion controller — the buttons send these strings to /motion_system/command but handlers must be implemented in motion_controller_reactive.cpp separately from the action-server path.

- FAILED state highlights in place, not at a distinct node — When robot_state is FAILED, the current stepper node turns red rather than advancing to a dedicated FAILED position. The diagram stays visually at whichever stage failed.

Motion Control & Planning

Purpose:
Receives detected objects from the perception subsystem and autonomously picks and places them into the correct bin based on material classification. The subsystem manages grasp strategy selection, collision-aware motion planning, gripper actuation, and drop-off pose handling.

Key topics / services / files / nodes

Name	Type	Role
Motion_controller *ros2 launch ur_gripper_demo ur_moveit.launch.py* \ ur_type:=ur3e \ onrobot_type:=rg2 \ demo_type:=reactive \ sim:=true	Node	Main motion planning and execution node.
Plastic_detections_translator	Node	Converts raw perception JSON data into typed ROS messages. Additionally, it helps account for environmental variables/mismatches.
test_helper python3 ~/git/Robotics-Studio-2/src/ur_gripper_demo/scripts/test_helper.py	Node	Injects synthetic detections for isolated subsystem testing.
ros2 service call /inject_detection std_srvs/srv/Trigger {}	Service Call	The service call for the test_helper node to send dummy data
ros2 service call /start_pick_place std_srvs/srv/Trigger {} Or ros2 topic pub --once /motion_system/command std_msgs/msg/String "data: 'START'"	Service Call	The service calls for starting the pick and place motion without the GUI.

Inputs:

/pick_queue

Raw sorted JSON list of detections from the OBB perception system containing:

- Object class and confidence
- Position (x, y, z) in mm
- Bounding box dimensions (dx, dy, dz)
- Orientation quaternion

Message type: JSON


```
[{
  "pose": {
    "position": {"x": 22.1, "y": -306.7, "z": -133.1},
    "orientation": {"qx": 0.0, "qy": 0.0, "qz": 0.71, "qw": 0.71}
  },
  "dimensions": {"dx_mm": 65.2, "dy_mm": 210.5, "dz_mm": 63.0},
  "classification": {"class": "pet_bottle", "confidence": 0.94},
  "debug": {
    "z_table_mm": -183.1,
    "z_approach_mm": -33.1,
    "angle_deg": 45.0,
    "angle_rad": 0.7854,
    "depth_m": 0.312,
    "dz_source": "depth"
  }
}]
```

/perception/objects

Typed ObjectArray messages used directly by the motion node:

- Pose in base_link
- Dimensions in metres
- Classification label
- Frame corrections (Y inversion and TCP Z offset)

Message Type: Custom Object_msg and Array of Object_msg:

{

std_msgs/Header header

geometry_msgs/Pose pose

string classification

float64[3] dimensions

}

/motion_system/command

String-based operator commands:

- START - Begins the pick-and-place sequence using the latest object detections
- STOP - Stops the active sequence
- HOME- Returns the robot to the home configuration and opens the gripper
- LOAD_BINS - Reloads bin drop-off poses from **bin_poses.json**
- SAVE_BIN:<name> - Saves the current arm pose as a named bin drop-off pose for the <name> classification

/start_pick_place

A trigger service is also available as a programmatic alternative to the GUI START command.

Bin drop-off poses are loaded from bin_poses.json during startup and matched according to the detected classification string.

Outputs:
To GUI / Monitoring

/motion_system/status

Publishes textual status updates such as planning progress and errors

/motion_system/current_object

Publishes the currently targeted object classification

To Gripper

/finger_width_controller/commands

Publishes a Float64MultiArray containing the desired RG2 finger separation in metres
0.000 m = fully closed
0.110 m = fully open

/gripper_action_controller

Publishes a control_msgs/action/GripperCommand that is:

"{command: {position: 0.11, max_effort: 20.0}}"

position: the width of the gripper opening in metres.
max_effort: the max force the gripper will apply before stalling, in Newtons.

To MoveIt Planning Scene
Collision objects are dynamically added and removed throughout operation:

- Ground plane and support platform primitives are spawned during startup
- Camera tripod collision geometry is spawned during startup
- Per-object collision boxes are inserted before grasping and removed after drop-off

Planning Requests

- Start and Goal state of robot
- Execute command

Key files:

- **motion_controller.cpp**
  Main motion node containing grasp logic, MoveIt planning, and gripper control.
- **scripts/plastic_detections_translator.py**
  Handles unit conversion and coordinate frame correction between perception and motion subsystems.
- **scripts/test_helper.py**
  Standalone detection injector used for subsystem standalone testing.
- **config/bin_poses.json**
  Stores drop-off poses for each material classification using Cartesian position and RPY orientation values.
- **meshes/UR3eTrolley_decimated.dae**
  Optional collision mesh for trolley representation.

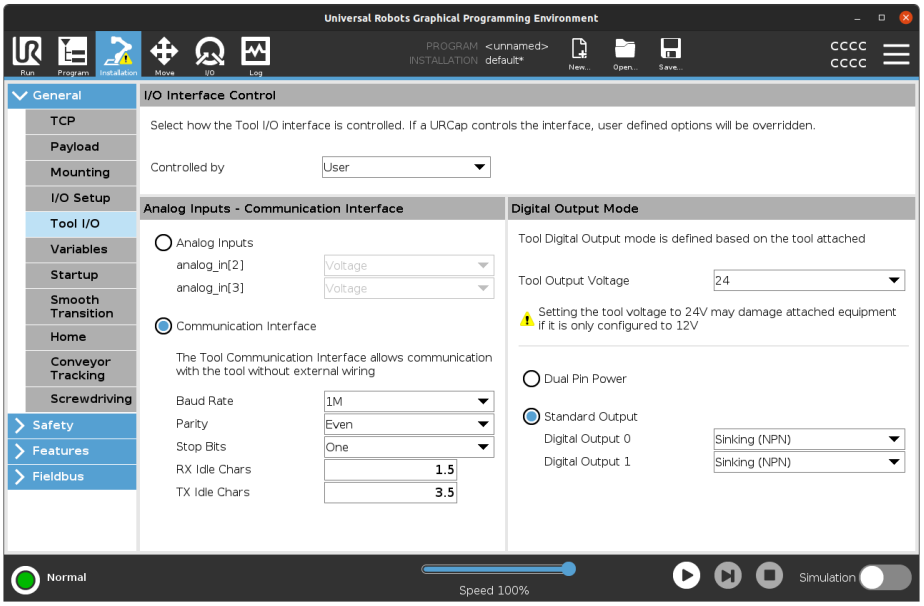
SubSystem Configuration		
Motion Control & Plan; Parameters defined in pick_n_place_try2_1.cpp		
Parameter	**Value**	**Description**
g_sim_mode	False	When true, closeGripperForGrasp() skips stall detection and always returns grasped. Useful for testing with fake_hardware
PREGRASP_HEIGHT	0.03 m	Vertical offset above the object used before descent.
SAFE_Z_HEIGHT	0.20 m	Clearance height used during transit motions.
GRIPPER_OPEN	0.110 m	Maximum RG2 finger separation.
GRIPPER_CLOSED	0.010 m	Nominal fully closed finger separation.
TOP_DOWN_MAX_SPAN	0.090 m	Objects narrower than this threshold are considered suitable for top-down grasping.
TOP_DOWN_MAX_HEIGHT	0.080 m	Objects shorter than this threshold are considered suitable for top-down grasping.
SIDE_GRASP_PITCH	25 °	Approach pitch angle used during slide-under grasping.
RG2_FINGER_MARGIN	0.0 m	Optional safety margin subtracted from grip width.
ATTEMPTS	2	The number of attempts to make when grasping single object
Translator		
MIN_CONFIDENCE (depreciated)	0.50 %	Detections below this threshold are discarded.
GRIPPER_TCP_OFFSET_M (depreciated)	0.218 m	Vertical offset between tool0 and gripper_tcp, derived from the URDF.
MoveIt Planning Configuration		
Planner	BiTRRTkConfig Default	The Planner used for trajectory generation
Planning time per attempt	3.0 s	Time allocated per planning attempt

SubSystem Configuration		
Number of candidate plans generated per move:	8	When going to a new goal state, how many separate plans to generate in order to find the most efficient.
Velocity scaling factor	0.3	
Acceleration scaling factor:	0.3	

Grasp Strategy Selection: The subsystem automatically selects a grasp strategy based on object geometry.		
Strategy	**Condition**	**Description**
TOP_DOWN	min(dx, dy) ≤ 90 mm height ≤ 80 mm	Vertical descent directly onto the object.
SIDE_HORIZONTAL	Object height greater than 80 mm Object is not thin (the thinnest width is over 3 cm)	Horizontal approach typically used for upright bottles.
SIDE_VERTICAL	Wide and relatively flat objects	Side approach intended to slide fingers underneath the object.

Set-up

1) **Power on the Robot**
2) **Connect to the robot via an Ethernet cable**
3) **Check your IP by typing hostname -I in your terminal**
4) **RG2 Gripper Setup**
    - **a)** Connect the OnRobot Quick Changer to the Tool I/O port of the UR3e robot.
    - b) On the teach pendant, navigate to Installation → General → Tool I/O and configure the following parameters:



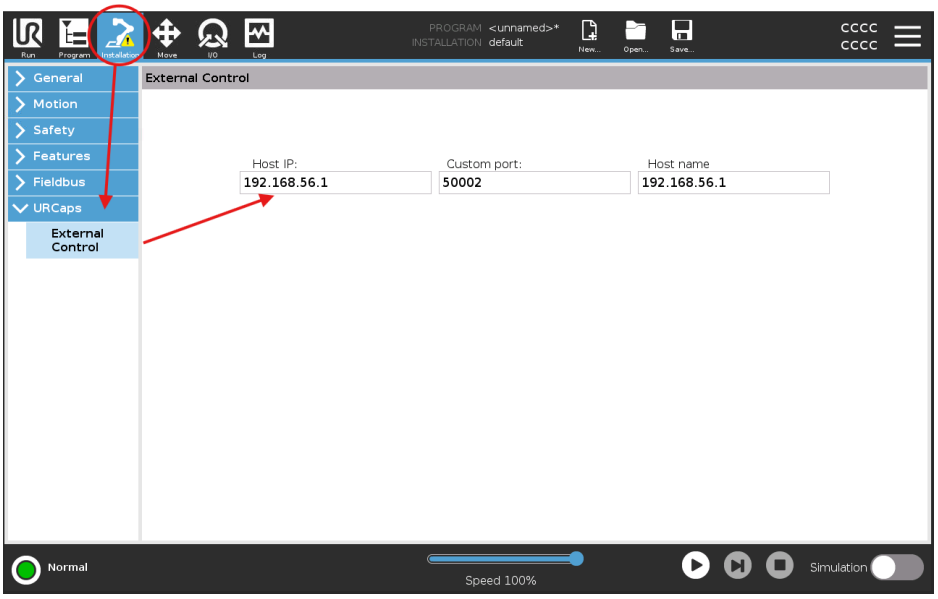
- Controlled by: User
- Communication Interface:
- Baud Rate: 1M
- Parity: Even
- Stop Bits: One
- RX Idle Chars: 1.5
- TX Idle Chars: 3.5
- Tool Output Voltage: 24V
- Standard Output:
- Digital Output 0: Sinking (NPN)
- Digital Output 1: Sinking (NPN)
- Usage

c) Still in the Installation tab, scroll down to URCaps → OnRobot and set the connection to "No Connection".

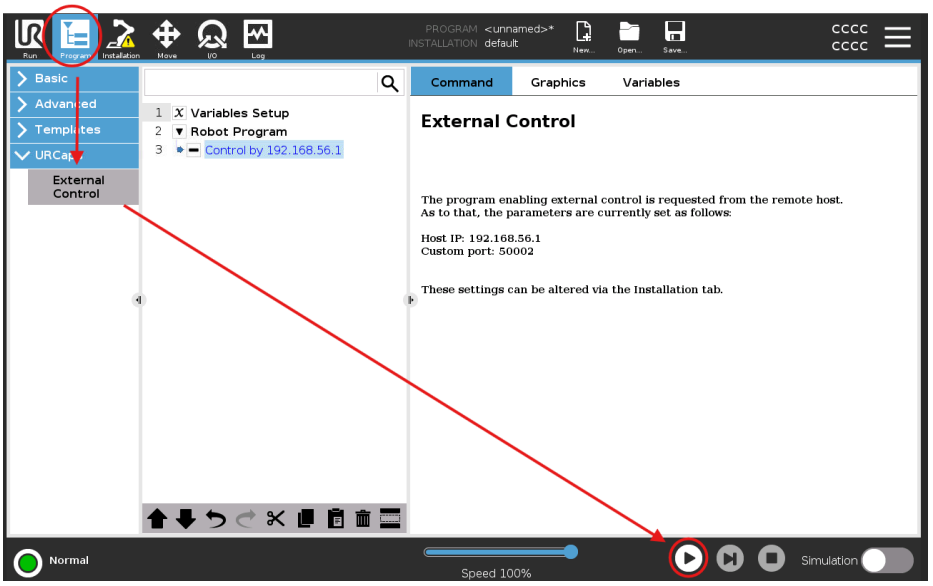
5) Setting up External Control on teach pendant and verify connection:

a. On the teach pendant, from the menu on the top of the screen, go to the Installation tab, scroll down to URCaps -> External Control and change the

Host IP to your device IP.



b. Afterwards go back to the Program tab and add External Control to the list and press play.



If no errors appear, the robot is ready to receive commands from the computer.

Running the Subsystem

Start the Driver:

Start the Subsystem:

ros2 launch ur_gripper_demo ur_moveit.launch.py demo_type:=motion_controller_test ur_type:=ur3e

Or:

ros2 launch ur_gripper_demo ur_moveit.launch.py demo_type:=motion_controller_test ur_type:=ur3e sim:=true

Sim=true argument turns off the gripper stall detection and assumes that grasp is always successful, this is due to the lack of contact forces in the simulation.

First time step:
Using freedrive from the teach pendant of the robot, move the robot to a desired drop off pose for a classification bin and set it either through the GUI or in the terminal where the motion controller is running by typing pose <class> . Do the same for every object classification.

Start:
Press Play in the Teach pendant

Standalone Testing

The **test_helper** node injects synthetic detections using the same format as the production perception system.
Terminal #1 (main node):

```
ros2 launch ur_gripper_demo ur_moveit.launch.py \
  ur_type:=ur3e \
  onrobot_type:=rg2 \
  demo_type:=reactive \
  sim:=true
```

Terminal #2 (helper node):

chmod +x test_helper.py

python3 ~/git/Robotics-Studio-2/src/ur_gripper_demo/scripts/test_helper.py

Terminal #3 (trigger helper node):

ros2 service call /inject_detection std_srvs/srv/Trigger {}

Pick and place test:

The test helper node spawns 3 different objects of different classifications at random locations on the tray. If an object is too close to the robot the robot will skip over that object after multiple attempts. With “sim:true” the system will bypass the grasp verification and drop monitoring. If testing those two features are being tested it is better to set sim:false.

Current Limitations and Future Improvements

- Static frame correction: The translator currently applies a fixed Y-axis inversion and TCP Z-offset (**`218 mm`**) to align perception data with the robot **`base_link`** frame.This is no longer needed as there are planned improvements in the camera calibration technique.
- Single-object execution pipeline — Objects are processed sequentially. Future work aims to support dynamic scene updates so removed or moved objects are immediately reflected in the planning scene.
- Side grasp strategies under development:
    - **`SIDE_VERTICAL`** is intentionally overridden for safety to avoid potential collisions between the robot and the table during slide-under motions.

    - **`SIDE_HORIZONTAL`** exists but is still being refined for reliable execution and is overridden in real life tests as it is assumed that the plastic objects are always on their sides.

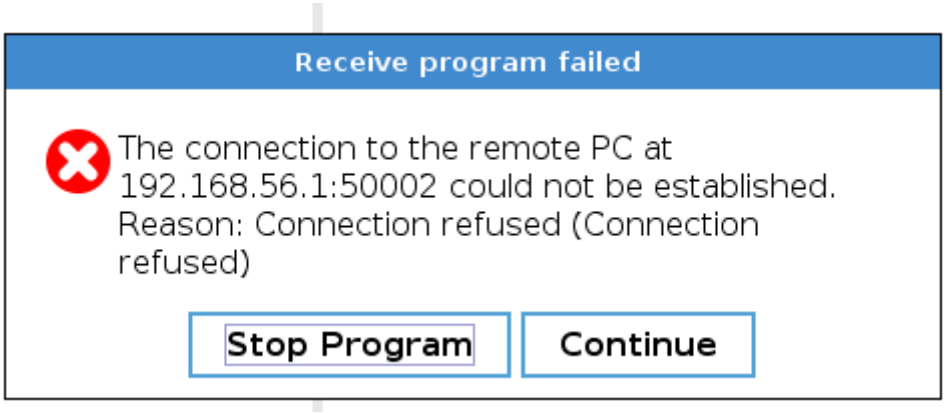
- Approximate collision environment: Static collision geometry (tripod, platform, ground) is manually positioned. Any physical layout changes require updating constants in motion_controller.cpp.
- The current grasp-verification mechanism relies solely on the measured gripper finger width to infer whether a grasp has succeeded. This limitation arises because, although the simulated onrobot_rg2 exposes a gripper_action controller capable of reporting states such as stalled or goal reached, the physical hardware does not provide equivalent interfaces. The real RG2, driven via RS-485, supports only position (finger-width) control and does not allow specifying or regulating grasping force. As a result, the system cannot perform force-based grasp verification or detect conditions such as object slip, insufficient grip, or early contact. Consequently, the current implementation may apply excessive force during closure, potentially crushing, deforming, or damaging non-rigid or fragile objects, since no feedback mechanism exists to modulate or limit applied effort.
- Grasp only occurs around the centroid of the object therefore thin thick hard and soft parts of the object are not considered in the grasp strategy.

Troubleshooting & FAQs

Common issues and fixes

WSL 2 Cannot Connect to the Robot

If you are using WSL 2 and experience connection issues with the robot driver, you may encounter errors after starting the driver and pressing Play on the teach pendant, such as:



- Connection not found
- Network communication errors between the robot and ROS/driver
- The robot driver failing to establish a socket connection

This issue commonly occurs because the Windows network interface is not properly exposed to WSL 2.

Verifying the Network Configuration

1. Open a WSL terminal and run:
   hostname -I
2. On Windows, open Command Prompt and run:
   ipconfig
3. Compare the IPv4 addresses from both outputs.
   - If the WSL environment shows the same network range as the Windows host, networking is likely configured correctly.
   - If WSL only displays internally generated addresses such as:
     172.xx.x.x

then WSL is using its default NAT-based virtual network and may not be able to communicate with the robot directly.

Fix: Enable Mirrored Networking in WSL

To expose the host network interface directly to WSL, enable mirrored networking.

1. Open or create the following file on Windows:
   C:\Users\<YourUsername>\.wslconfig
2. Add the following configuration:
   [wsl2]
   networkingMode=mirrored
   dnsTunneling=true
   autoProxy=true
3. Restart WSL by running the following command in PowerShell or Command Prompt:
   wsl --shutdown
4. Reopen WSL and verify the IP address again using:
   hostname -I

WSL should now share the same network visibility as the Windows host, allowing communication with the robot driver.

Motion Plan & Controller

1. Grasp confirmation false-negative (object held but flagged as dropped) The gripper closes against a soft or deformable object. If the compliance of the object is high enough that the gripper continues to close past the stall force threshold, `reached_goal` fires instead of `stalled`.

Fix: lower `max_effort` in the grasp goal and tune it to the material properties of the expected objects. Stiffer objects tolerate higher effort; foam or fabric requires a softer touch.

2. MoveIt planning failure after grasp (invalid goal) After picking up an object, the combined arm+object collision geometry may make the bin pose unreachable — particularly if the bin is close to the robot base, which forces the wrist into a folded configuration outside the tightened joint limits.

Fix: when teaching bin poses via freedrive, observe the joint state readout on the teach pendant and confirm that wrist_1_joint is within [0°, –180°] and elbow_joint is within [–135°, 135°]. If planning consistently fails for a bin, the bin pose should be adjusted to a more reachable configuration.

3. "No bin defined for class: X" A classification label received from perception has no associated drop-off pose in `bin_poses.json`.

Fix: freedrive the robot to the desired bin pose and save it using the GUI button or by typing `pose <class_name>` in the terminal. The pose is saved to the JSON file immediately and takes effect on the next run.

Grasp Confirmation

The gripper is commanded to close to 5 mm with a specified effort. The GripperActionController reports stalled = true if the gripper fingers contact the object before reaching the target position. This stall condition is used as the grasp confirmation signal. If reached_goal = true and stalled = false, the gripper has fully closed without contacting an object, indicating a failed grasp attempt, and the operation is retried.Grasp Confirmation

The gripper is commanded to close to 5 mm with a specified effort. The GripperActionController reports stalled = true if the gripper fingers contact the object before reaching the target position. This stall condition is used as the grasp confirmation signal. If reached_goal = true and stalled = false, the gripper has fully closed without contacting an object, indicating a failed grasp attempt, and the operation is retried.

Appendices

Appendix 1: How the Motion Subsystem Works

The subsystem receives, from the perception node, the pose of each detected object (position and orientation in the robot base frame), its classification (material type — HDPE bottle, metal, fabric, etc.), and the dimensions of its axis-aligned bounding box.

The bounding box serves two purposes. First, it allows a physical collision representation of the object to be registered in the MoveIt planning scene before robot motion begins, ensuring that path planning accounts for the object's full volume rather than only its centroid. This is important because the detected objects are typically larger than the gripper fingers, and ignoring their geometry could result in planned trajectories intersecting the object itself. Second, the bounding box dimensions are used for grasp strategy selection.

Grasp Strategy Selection

Given the bounding box dimensions `[dx, dy, dz]`:

- **TOP_DOWN** — selected when the narrowest XY dimension is less than 90 mm (the gripper maximum span) and the object height is below 80 mm. The gripper approaches vertically and descends onto the object's narrow axis.
- **SIDE_HORIZONTAL** — selected when the object is tall (`height > 80 mm`) and not extremely thin. This strategy targets upright bottles and similar objects, where the gripper approaches horizontally and encloses the object cross-section.
- **SIDE_VERTICAL** — selected for wide, flat objects where neither of the previous strategies is appropriate. The gripper is tilted and inserted beneath the object before lifting. This strategy is implemented and validated in simulation but excluded from the physical demonstration due to the risk of gripper finger contact with the table surface.

Execution Sequence

The system resolves all detected objects, sorts them by destination bin to minimise travel, and processes them sequentially.

1. **Pregrasp**
   The robot moves to a standoff pose appropriate for the selected strategy — above the object for `TOP_DOWN`, or offset horizontally for `SIDE_HORIZONTAL`.
2. **Approach**
   A Cartesian motion moves the gripper from the pregrasp pose to the final grasp pose.
3. **Attach**
   Before closing the gripper, the object is registered as attached to the end effector within the MoveIt planning scene. From this point onward, the object's collision geometry moves with the robot and is considered during motion planning, preventing the object from intersecting obstacles during transport.
4. **Grasp Confirmation**
   The gripper is commanded to close to 5 mm with a specified effort. The `GripperActionController` reports `stalled = true` if the gripper fingers contact the object before reaching the target position. This stall condition is used as the grasp confirmation signal. If `reached_goal = true` and `stalled = false`, the gripper has fully closed without contacting an object, indicating a failed grasp attempt, and the operation is retried.
5. **Transport and Drop Monitoring**
   Once grasped, the robot plans and executes a trajectory to the destination bin. During transport, a background thread re-sends the same gripper close command every 500 ms. If the object is dropped, the fingers close fully (`reached_goal = true`, `stalled = false`), indicating loss of contact. The current perception snapshot is then discarded, a fresh perception update is requested, and the grasp sequence is retried.
6. **Release**
   At the destination bin, the gripper opens, the object is detached from the planning scene, and the collision object is removed.
7. **Return Home**
   After all objects have been processed, the robot returns to the UR3e up named pose.